

Hardware-Accelerated Matrix Multiplication on Zynq-7000 FPGA

This project implements a progressively optimized hardware-software co-design for matrix multiplication on the Zybo Z7 FPGA platform using the Xilinx Zynq-7000 architecture.

The project accelerates matrix multiplication by offloading computation from the ARM Cortex-A9 Processing System (PS) into a custom FPGA accelerator implemented in the Programmable Logic (PL).

Final architecture highlights:

- 64×64 matrix multiplication
- 32×32 tiled execution
- int8 inputs
- int32 accumulation
- 64-bit AXI-Stream DMA
- BRAM-based tile buffers
- Ping-pong buffering
- 16-column parallel MAC datapath
- True overlapped load + compute scheduling

Final measured performance:

- Software time: ~10073 us
- Hardware time: ~1952 us
- Final speedup: ~5.15×

System Architecture

The final architecture contains two major domains:

1. Processing System (PS)

- ARM Cortex-A9
- DDR3 memory
- AXI Timer
- Software control and verification

2. Programmable Logic (PL)

- AXI DMA
- AXI-Lite register interface
- BRAM tile buffers
- Ping-pong A/B buffering
- Controller FSM
- 16-column PE array

Dataflow:

DDR → AXI DMA MM2S → Accelerator → AXI DMA S2MM → DDR

Phases 1–8: Foundation Development

These phases established the basic hardware-software infrastructure required for the final accelerator.

Phase 1:

Pure software matrix multiplication reference implementation.

Phase 2:

Simulation of a single multiply-accumulate Processing Element (PE).

Phase 3:

Zybo board bring-up and ARM execution verification.

Phase 4:

Software matrix multiplication on ARM Cortex-A9.

Phase 5:

AXI-Lite register interface for ARM ↔ PL communication.

Phase 6:

Small FSM-based PL matrix multiplier without DMA.

Phase 7:

AXI DMA loopback verification.

Phase 8:

Streaming pass-through accelerator validating AXI-Stream handshaking.

Phase 9 — First Real DMA Matrix Multiply

Goal:

Implement first complete DMA-driven matrix multiplication.

Configuration:

- 4×4 matrix multiply
- 32-bit AXI-Stream
- Single compute datapath

Features:

- AXI DMA integration
- AXI-Lite control
- Hardware/software verification

Result:
MATRIX MULTIPLY 4x4 PASS

Phase 10 — Scaling Matrix Dimensions

Goal:
Scale matrix dimensions beyond 4x4.

Tested:
- 8x8
- 16x16

Changes:
- Larger memories
- Expanded counters
- Larger DMA buffers

Result:
MATRIX MULTIPLY 16x16 PASS

Phase 11 — 8 Parallel Processing Elements

Goal:
Increase compute throughput using parallel MAC datapaths.

Architecture:
- 8 output columns computed simultaneously
- 8 accumulators
- Broadcast A[i][k]
- Parallel B reads

Result:
8-PE MATRIX MULTIPLY PASS

Limitation:
Scaling caused LUT over-utilization because memories were still distributed logic.

Phase 12 — BRAM-Based Tile Buffers

Goal:
Replace LUT/register memories with BRAM storage.

Architecture:

- BRAM A tile buffer
- Banked B BRAMs
- Banked C accumulator BRAMs

Benefits:

- Efficient parallel reads/writes
- Reduced LUT usage
- Larger matrix support

Result:

32×32 BRAM MATRIX MULTIPLY PASS

Phase 13 — 64×64 Software-Controlled Tiling

Goal:

Support matrices larger than tile size.

Configuration:

- FULL_SIZE = 64
- TILE = 32

Software performed tiled execution:

```
for tile_i
  for tile_j
    for tile_k
```

PL computed one 32×32 tile multiply per invocation.

Initial Performance:

Software: 10048 us

Hardware: 14879 us

Speedup: 0.67×

Reason hardware was slower:

- Tile extraction overhead
- Packing overhead
- DMA setup overhead
- Software accumulation
- UART prints

Result:

TILED MATRIX MULTIPLY 64×64 PASS

Phase 14 — PL-Side K-Tile Accumulation

Goal:

Move partial accumulation from PS into PL.

Previous:

PS accumulated partial sums in software.

New:

PL internally accumulated:

$C_tile += A_tile_k \times B_tile_k$

NUM_KTILES register introduced.

Performance:

Software: 10072 us

Hardware: 8760 us

Speedup: 1.14×

Result:

PL-SIDE KTILE ACCUMULATION 64x64 PASS

Phase 15 — Pre-Packed Tile Streams

Goal:

Move tile extraction and packing outside the timed region.

Optimization:

- Pre-pack all tile streams before timing
- Timed loop only performs DMA + compute

Major Insight:

Removing UART prints, extraction, and packing dramatically improved performance.

Performance:

Software: 10049 us

Hardware: 3178 us

Speedup: 3.16×

Result:

PRE-PACKED TILE INPUT 64x64 PASS

Phase 16 — 64-Bit AXI-Stream

Goal:

Upgrade AXI-Stream from 32-bit to 64-bit.

Changes:

- 8 int8 values per beat
- 2 int32 outputs per beat
- 64-bit DMA streams
- u64 software buffers

Additional FSM states required for dual-output BRAM reads.

Performance:

Software: 10072 us

Hardware: 3293 us

Speedup: 3.05×

Result:

PHASE 16 64-BIT STREAM 64x64 PASS

Phase 17A — Ping-Pong Buffer Infrastructure

Goal:

Introduce double-buffered A/B memories.

Changes:

- A0/A1 tile buffers
- Duplicated B banks
- Buffer selection logic

Purpose:

Prepare infrastructure for overlapped execution.

Result:

PASS with similar timing.

Phase 17C — Faster Output Packing

Goal:

Reduce output FSM overhead.

Optimization:

Adjacent output values already resided in adjacent C banks.

Both values could therefore be fetched from the same address cycle.

Benefits:

- Reduced FSM states
- Reduced output latency

Performance:

Software: 10073 us

Hardware: 3171 us

Speedup: 3.17×

Result:

PHASE 17C 64-BIT STREAM 64x64 PASS

Phase 18A — 16-Column Parallelism

Goal:

Increase compute parallelism from 8 columns to 16 columns.

Changes:

- 16 B banks
- 16 C accumulator banks
- 16 parallel MAC datapaths

This phase produced the largest single architectural speedup.

Performance:

Software: 10073 us

Hardware: 2136 us

Speedup: 4.71×

Result:

PHASE 18A 64-BIT STREAM 64x64 PASS

Phase 18B — True K-Tile Prefetch Overlap

Goal:

Overlap tile loading with compute execution.

Behavior:

- K-tile 0 loaded into buffer 0
- K-tile 1 prefetched into buffer 1 during compute
- Compute swaps directly to buffer 1

Added:

- Independent prefetch FSM
- Prefetch synchronization logic
- Overlap scheduler states

Performance:

Software: 10073 us

Hardware: 1952 us

Speedup: 5.15×

Result:

PHASE 18B 64-BIT STREAM 64x64 PASS

Phase 18C — Final Stable Optimized Architecture

Phase 18C represents the final stable architecture integrating all successful optimizations.

Final Features:

- AXI DMA streaming
- 64-bit AXI-Stream
- BRAM tile storage
- Ping-pong buffering
- 16-column parallel MAC datapath
- Fast output packing
- True overlapped prefetch + compute scheduling

Final Performance:

Software: 10073 us

Hardware: 1952 us

Speedup: 5.15×

The final architecture achieved all major project goals.

Final Performance Summary

Phase	Software Time	Hardware Time	Speedup
Phase 13	10048 us	14879 us	0.67×
Phase 14	10072 us	8760 us	1.14×
Phase 15	10049 us	3178 us	3.16×
Phase 16	10072 us	3293 us	3.05×
Phase 17C	10073 us	3171 us	3.17×
Phase 18A	10073 us	2136 us	4.71×
Phase 18B/18C	10073 us	1952 us	5.15×

Conclusion

The project evolved from a simple software matrix multiplication implementation into a highly optimized FPGA accelerator featuring:

- AXI DMA streaming
- tiled execution
- BRAM tile buffering
- parallel PE arrays
- 64-bit streaming
- ping-pong buffering
- overlapped prefetch scheduling

The final accelerator achieved approximately 5.15× speedup over the ARM software baseline while maintaining full correctness.